

ADSO

Training 5

Manteniment del sistema de fitxers

Índex

1. Introducció.....	2 1.1.
Objectiu.....	3
2. Abans de començar.....	3 3.
Partició per emmagatzemar les còpies de seguretat.....	3 Quina modificació és necessària fer perquè aquesta nova partició es munti automàticament durant el boot amb mode de només lectura?.....4
3. Realització de còpies amb TAR.....	4 3.1.
Realització de còpies completes.....	4 3.2.
Realització de còpies incrementals.....	6 3.3.
Restauració d'una còpia de seguretat.....	8 3.4.
Restauració d'un fragment.....	9
4. Realització de còpies usant RSYNC.....	9 4.1.
Realització de backups a través d'una xarxa.....	10 \$ echo "nou arxiu" > /root/arxiu_nou.txt.....10 4.2.
Realització de còpies incrementals inverses.....	11 4.3.
Realització de còpies incrementals inverses tipus snapshot.....	13 Repàs d'enllaços durs.....13 4.3.1
Backups tipus "snapshot" amb rsync i cp -al.....	14 4.4. Script per fer backups tipus snapshot.....15
5. Referències.....	17 6.
Apèndix. Codi de l'script per còpies tipus snapshot.....	17

1

1. Introducció

Una de les tasques més importants de l'administrador de sistemes és la realització de còpies de seguretat que permeten restaurar el sistema complet en una quantitat acceptable de temps quan es produeix una fallada del sistema amb pèrdua de dades. Aquestes pèrdues poden ser degudes a múltiples factors com poden ser fallades de hardware, de software, accions humanes (accidentals o premeditades) o desastres naturals.

Abans de començar a realitzar còpies de seguretat l'administrador del sistema ha de decidir una política tenint en compte aspectes com:

- Seleccionar el tipus correcte de medi físic per fer les còpies de seguretat tenint en compte la grandària, el cost, la velocitat, la disponibilitat, l'usabilitat i la confiabilitat.
- Decidir quins fitxers necessiten una còpia de seguretat i on són aquest fitxers. Són més importants el fitxers de configuració del sistema i els fitxers dels usuaris (normalment ubicats a /etc i /home respectivament) que el fitxers temporals o els binaris del sistema (/tmp i /bin)
- Decidir la freqüència i el tipus de planificació de les còpies de seguretat. Això depèn de la variabilitat de les dades. Una base de dades pot necessitar múltiples còpies de seguretat diàries, mentre que un servidor web pot requerir només una còpia diària i altres sistemes de fitxers poden requerir només una còpia setmanal.

– Analitzar altres aspectes com: on s'han d'emmagatzemar les còpies, per quan temps s'han de mantenir i amb quina rapidesa es necessita poder recuperar cada tipus de fitxer.

Utilitzant tota la informació anterior es pot decidir finalment una estratègia de còpies de seguretat. Això inclou decidir la freqüència de les còpies i el tipus. Una estratègia comú es fer còpies completes i còpies incrementals. D'aquesta manera es pot disminuir el temps i la grandària de les transferències de dades de les còpies però també s'incrementa la complexitat de la restauració de les dades.

Una estratègia típica consisteix en realitzar còpies completes (també conegudes com de nivell 0) setmanalment i còpies incrementals (conegudes com de nivell 1 o més gran) diàriament. Si el

2

grau de variabilitat dels fitxers és molt gran es pot modificar l'anterior model setmanal per un model mensual on cada més es realitza una còpia de nivell 0, cada setmana una còpia de nivell 1 (incremental setmanal sobre el nivell 0) i cada dia es fa una còpia de nivell 2 (incremental diari sobre el nivell 1).

Per últim s'han de decidir les eines més adequades per implementar l'estratègia de còpies de seguretat que s'ha dissenyat. Com a primer pas en aquest objectiu usarem el mateix disc dur com medi físic per fer les còpies de seguretat tot i que no és el més convenient habitualment a causa del alt risc de que una pèrdua de dades del disc afecti també a les còpies de seguretat. Per fer les còpies utilitzarem dos tipus diferents d'aplicacions de còpia de seguretat: **tar**, que realitza les còpies a través del sistema de fitxers i **rsync** que permet sincronitzar discs amb moltes opcions de configuració i per tant permet implementar diferents estratègies de còpies de seguretat.

1.1. Objectiu

Aprendre a dissenyar i implementar sistemes de còpies de seguretat tot utilitzant eines bàsiques de UNIX.

2. Abans de començar

contesteu les següents preguntes abans de començar:

1. Com es pot empaquetar i desempaquetar un(s) fitxer(s) utilitzant la comanda tar?

La comanda **tar** (Tape ARchive) s'utilitza per agrupar múltiples fitxers en un de sol, mantenint permisos i estructura de directoris.

Per empaquetar (Crear un arxiu): S'utilitzen les opcions -cvf (Create, Verbose, File).

Bash

None

```
tar -cvf nom_arxiu.tar /ruta/del/directori_o_fitxers
```

-c: Crea un nou arxiu.

-v: Mostra el procés en pantalla (verbose).

-f: Permet especificar el nom del fitxer resultant.

Per desempaquetar (Extreure un arxiu): S'utilitzen les opcions -xvf (Extract, Verbose, File).

Bash

None

```
tar -xvf nom_arxiu.tar
```

-x: Extreure el contingut de l'arxiu.

2. Què és un enllaç dur (hard link)? I quina diferència hi ha entre fer una còpia (cp file_a file_b) i fer un hard link (ln file_a file_b)?

Per entendre això, cal recordar que per a Linux, el nom d'un fitxer no és el fitxer en si mateix, sinó només una "etiqueta" que apunta a un número d'identificació de les dades al disc, anomenat **inode**⁴. Què és un Hard Link?

Un enllaç dur és simplement un segon nom que apunta exactament al mateix inode que el fitxer original. Això significa que el fitxer (les dades físiques) té ara dos noms vàlids.

Diferència entre cp i ln:

Característica	Copia (cp file_a file_b)	Enllaç Dur (ln file_a file_b)
Inode	Crea un nou inode (nou número ID).	Comparteix el mateix inode ⁵ .
Espai en disc	Ocupa el doble d'espai (duplica les dades).	No ocupa espai extra (només una entrada de directori).
Modificacions	Si canvies file_a, file_b NO canvia. Són independents.	Si canvies file_a, file_b TAMBÉ canvia, perquè són les mateixes dades.
Esborrat	Si esborres l'original, la còpia es manté intacta.	Si esborres l'original, les dades no s'esborren fins que s'eliminin tots els enllaços durs (comptador d'enllaços a 0).

3. Partició per emmagatzemar les còpies de seguretat. Creeu una nova partició a l'espai lliure que teniu al disc i doneu-li format extended3. Munteu aquesta partició sobre el directori /backup de forma que tan sols root tingui permisos d'accés. La

3

resta d'usuaris no han de tenir ni tan sols permís de lectura al directori ja que el contingut de les còpies de seguretat podria ser informació confidencial.

Quines comandes heu utilitzat per crear la partició, donar format al sistema de fitxers, muntar la partició i canviar els permisos del directori backup?

Entrem fdisk.

```
root@davidP (dl. de des. 08):~# fdisk /dev/sda

Welcome to fdisk (util-linux 2.41).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

This disk is currently in use - repartitioning is probably a bad idea.
It's recommended to umount all file systems, and swapoff all swap
partitions on this disk.

Command (m for help): █
```

Creem partició.

```

Command (m for help): n
Partition type
   p   primary (2 primary, 1 extended, 1 free)
   l   logical (numbered from 5)
Select (default p): p

Selected partition 2
First sector (53688320-67108863, default 53688320):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (53688320-67108863, default 67108863): +100M

Created a new partition 2 of type 'Linux' and of size 100 MiB.

Command (m for help): w
The partition table has been altered.
Syncing disks.

root@davidP (dl. de des. 08):~#

```

Donem sistema de fitxers ext3.

```

root@davidP (dl. de des. 08):~# mkfs -t ext3 /dev/sda2
mke2fs 1.47.2 (1-Jan-2025)
Creating filesystem with 102400 1k blocks and 25584 inodes
Filesystem UUID: a73f0c81-bd05-476a-aac8-4164dc18544a
Superblock backups stored on blocks:
    8193, 24577, 40961, 57345, 73729

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

```

Donem permisos.

```

root@davidP (dl. de des. 08):/home/homeA# mkdir backup
root@davidP (dl. de des. 08):/home/homeA# chown root:root backup/
root@davidP (dl. de des. 08):/home/homeA# chmod 700 backup/
root@davidP (dl. de des. 08):/home/homeA# ls -la
total 28
drwxr-xr-x  4 root root  4096  8 de des.  20:27 .
drwxr-xr-x 11 root root  4096 17 de nov.  19:10 ..
drwx-----  2 root root  4096  8 de des.  20:27 backup
drwx-----  2 root root 16384  3 d'oct.  11:20 lost+found

```

Montem el directori al sda2

```

root@davidP (dl. de des. 08):/home/homeA# mount /dev/sda2 backup/
root@davidP (dl. de des. 08):/home/homeA# mount -o remount,rw /dev/sda2 backup/

```

Per afegir més proteccions a aquest directori es pot muntar en mode de escriptura només quan s'escriguin els backups i la resta del temps muntar-lo en mode de només lectura. Normalment, s'hauria de desmuntar i tornar a muntar canviant les opcions per defecte. Però és possible canviar les opcions d'una partició sense desmuntar-la si feu servir l'opció **remount**.

Muntar només per lectura:

```
$ mount -o remount,ro /dev/usb4 /backup
```

Muntar per lectura i escriptura:

```
$ mount -o remount,rw /dev/usb4 /backup
```

Quina modificació és necessària fer perquè aquesta nova partició es munti automàticament durant el boot amb mode de només lectura?

Modificar el /etc/fstab.

GNU nano 8.4 /etc/fstab *					
#<dispositiu>	<directori>	<tipus>	<parametres>	<dump>	<pass>
/dev/sda1	/	ext4	defaults	0	1
/dev/sda5	/usr/local	ext4	defaults	0	2
/dev/sda6	/home/homeB	ext4	defaults	0	2
/dev/sdb1	/home/homeA	ext4	defaults	0	2
/dev/sdb3	none	swap	defaults	0	0
/dev/sda2	/backup	ext3	ro	0	2

3. Realització de còpies amb TAR

3.1. Realització de còpies completes

Realitzeu una còpia completa del directori /root (comproveu que existeixen fitxers en aquest directori) amb la comanda **tar**. Useu noms significatius per als fitxers de *backup*: feu que el nom del fitxer tingui informació sobre el contingut del fitxer, la data i hora en que es va fer la còpia, i si la còpia és completa o incremental, i el nivell de la còpia (0, 1, ...).

```
root@davidP (dl. de des. 08):/home/homeA# tar -cvzf backup/backup-root.tar.gz /root
root@davidP (dl. de des. 08):/home/homeA# ls -la backup/
total 90316
drwxr-xr-x 3 root root    1024 8 de des. 20:32 .
drwxr-xr-x 4 root root    4096 8 de des. 20:27 ..
-rw-r--r-- 1 root root 92102656 8 de des. 20:32 backup-root.tar.gz
drwx----- 2 root root    12288 8 de des. 20:25 lost+found
```

Com es pot fer perquè el nom del fitxer de *backup* inclogui automàticament la data del *backup*? per exemple que sigui backup-etc-nivell0-202112041030 (any mes dia hora minut segon). Utilitza la comanda **date**.

```
root@davidP (dl. de des. 08):/home/homeA# tar -cvzf backup/backup-root-nivell0-$(date +%Y%m%d%H%M%S).tar.gz /root
tar: Es treuen els «/» del començament dels noms dels membres
/root/
/root/.gnupg/
/root/.gnupg/private-keys-v1.d/
/root/.dmrc
/root/.selected_editor
/root/ocupacio.sh
/root/Plantilles/
/root/.xsession-errors.old
/root/class_act.sh
/root/BadUsers.sh
/root/infousers.sh
/root/Públic/
/root/Imatges/
/root/asosh-0.1.tar.gz
/root/net-out.sh
/root/.cache/
/root/.cache/mesa_shader_cache_db/
/root/.cache/mesa_shader_cache_db/part5/
/root/.cache/mesa_shader_cache_db/part5/mesa_cache.idx
/root/.cache/mesa_shader_cache_db/part5/mesa_cache.db
/root/.cache/mesa_shader_cache_db/part49/
/root/.cache/mesa_shader_cache_db/part49/mesa_cache.idx
/root/.cache/mesa_shader_cache_db/part49/mesa_cache.db
/root/.cache/mesa_shader_cache_db/part41/
```

```

root@davidP (dl. de des. 08):/home/homeA# ls -la backup/
total 90316
drwxr-xr-x 3 root root    1024 8 de des. 20:43 .
drwxr-xr-x 4 root root   4096 8 de des. 20:27 ..
-rw-r--r-- 1 root root      0 8 de des. 20:43 backup-root-nivell0-20251208204355.tar.gz
-rw-r--r-- 1 root root 92102656 8 de des. 20:32 backup-root.tar.gz
drwx----- 2 root root   12288 8 de des. 20:25 lost+found
root@davidP (dl. de des. 08):/home/homeA#

```

Quina comanda heu fet servir per fer la còpia completa del directori /etc?

```

root@davidP (dl. de des. 08):/home/homeA# tar -cvzf backup/backup-root-nivell0-$(date +%Y%m%d%H%M%S).tar.gz /etc
tar: Es treuen els «/» del començament dels noms dels membres
/etc/
/etc/snmp/
/etc/snmp/snmp.conf
/etc/init.d/
/etc/init.d/plymouth-log
/etc/init.d/anacron
/etc/init.d/sudo
/etc/init.d/lightdm
/etc/init.d/procps
/etc/init.d/lm-sensors
/etc/init.d/capod

```

Per què no és aconsellable comprimir el fitxer de *backup*?

Tot i que comprimir estalvia espai, hi ha raons tècniques per les quals es pot desaconsellar en certs entorns crítics:

Recuperació de dades (Integritat): Aquesta és la raó més important.

En un fitxer **.tar (sense comprimir)**, si una part del fitxer es corromp (per exemple, un sector defectuós al disc), normalment **encara pots recuperar la resta** de fitxers de l'arxiu, ja que les dades s'emmagatzemen seqüencialment.

En un fitxer **comprimit (.tar.gz o .tar.xz)**, si es corromp un sol bit, sovint **tot l'arxiu queda inutilitzable** o es perd tota la informació a partir del punt del dany, ja que l'algorisme de descompressió necessita la integritat del flux de dades.

Velocitat i ús de CPU: La compressió requereix potència de processament. Si estàs fent una còpia de seguretat d'un servidor amb molta càrrega o necessites que la còpia es faci el més ràpidament possible (limitat només per la velocitat del disc), és millor no comprimir.

Dades ja comprimides: Si el directori conté fitxers que ja estan comprimits (imatges JPEG, vídeos, fitxers .zip), tornar-los a comprimir amb tar amb prou feines estalviarà espai i malbaratarà temps de CPU.

Si volguéssim comprimir el fitxer de *backup* quina opció afegiríem a la comanda tar?

Si decidim que volem estalviar espai i assumir el cost de CPU, l'opció més comuna és utilitzar l'algorisme **gzip**.

L'opció que afegiríem és **-z**.

La comanda quedaria així:

```
tar -czvf backup_etc.tar.gz /etc
```

A vegades quan es realitza la còpia completa es requereix excloure certs fitxers. Per això es pot construir un fitxer amb una llista de fitxers que no haurien de ser al *backup*.

Feu de nou la copia completa però aquesta ocasió exclouent el fitxers que siguin al fitxer *excludes*. (poseu el nom d'alguns fitxers al fitxer *excludes*). Quina opció de **tar** permet excloure un llistat de fitxers del *backup*?

Creem archiu:

```

GNU nano 8.4                               excludes
cache/
*.tmp

```

I executem això.


```

root@davidP (dl. de des. 08):/home/homeA# tar -cvzf backup/backup-root-nivell0-$(date +%Y%m%d%H%M%S).tar.gz --exclude-from=excludes /etc
tar: Es treuen els «/» del començament dels noms dels membres
/etc/
/etc/snmp/
/etc/snmp/snmp.conf
/etc/init.d/
/etc/init.d/plymouth-log
/etc/init.d/anacron

```

5

A més a més de protegir el directori de les còpies de seguretat és importat utilitzar algun mecanisme que permeti verificar que el fitxers de *backup* no hagin estat modificats després d'haver-los creat. Per això es comú utilitzar algun mecanisme de signatura digital, com son els *hashs* SHA, que permeten verificar la integritat d'un fitxer.

Una vegada hagueu realitzat la còpia, utilitzeu la comanda **sha512sum** (utilitzeu el **man** per saber com fer servir aquesta comanda) amb la còpia del directori i guardeu vos el resultat en un fitxer <nomdelacopia>.asc.

```

root@davidP (dl. de des. 08):/home/homeA# sha512sum backup/backup-etc-nivell0-20251208205058.tar.gz > backup/backup-etc-nivell0-20251208205058.tar.gz.asc
root@davidP (dl. de des. 08):/home/homeA# ls -la backup/
total 90317
drwxr-xr-x 3 root root    1024 8 de des. 20:51 .
drwxr-xr-x 4 root root    4096 8 de des. 20:48 ..
-rw-r--r-- 1 root root      0 8 de des. 20:50 backup-etc-nivell0-20251208205058.tar.gz
-rw-r--r-- 1 root root    178 8 de des. 20:51 backup-etc-nivell0-20251208205058.tar.gz.asc
-rw-r--r-- 1 root root      0 8 de des. 20:43 backup-root-nivell0-20251208204355.tar.gz
-rw-r--r-- 1 root root      0 8 de des. 20:45 backup-root-nivell0-20251208204516.tar.gz
-rw-r--r-- 1 root root      0 8 de des. 20:49 backup-root-nivell0-20251208204901.tar.gz
-rw-r--r-- 1 root root 92102656 8 de des. 20:32 backup-root.tar.gz
drwx----- 2 root root   12288 8 de des. 20:25 lost+found

```

3.2. Realització de còpies incrementals

Per tal de dur a terme còpies incrementals, serà necessari modificar alguns fitxers dins el directori /root :

Modificacions:

- Genereu nous fitxers i subdirectoris
- Modifiqueu el contingut d'alguns fitxers
- Useu la comanda **touch** per canviar la data de modificació d'alguns fitxers.

Per fer còpies incrementals amb tar tenim l'opció **--newer** que només inclou els fitxers que hagin estat modificats des d'una data determinada. Aquesta data es pot especificar de dues formes: la primera, posant-la directament, per exemple: "**--newer="2021-11-28 12:10"**". La segona manera consisteix en agafar la data d'un fitxer, això vol dir que la data serà la de l'última

6

modificació del fitxer, per exemple: "**--newer=./file**".

Ara realitzeu una còpia incremental del directori /root respecte a la còpia completa que heu fet abans usant la comanda tar. Feu també un **sha512sum** i guardeu-lo en un arxiu.asc amb el nom diferent al nom que heu utilitzat abans.

Primera ronda de modificacions

```

root@davidP (dl. de des. 08):/home/homeA# mkdir /root/nou_directori_1
root@davidP (dl. de des. 08):/home/homeA# touch /root/nou_fitxer_1.txt
root@davidP (dl. de des. 08):/home/homeA# echo "Contingut nou per la prova" | tee -a /root/nou_fitxer_1.txt
Contingut nou per la prova
root@davidP (dl. de des. 08):/home/homeA# touch /root/.bashrc

```

Primera còpia incremental i Checksum

```

root@davidP (dl. de des. 08):/home/homeA# tar -cvf backup_incl.tar --newer=backup_completa.tar /root
root@davidP (dl. de des. 08):/home/homeA# sha512sum backup_incl.tar > backup_incl.asc

```

Quin problema potencial hi ha al utilitzar el fitxer .tar de la còpia completa per obtenir la data del backup per fer la còpia incremental? Com es pot resoldre aquest problema?

Resposta: El problema és el "**forat de temps**" (**time gap**). La data de modificació del fitxer .tar resultant correspon al moment en què **acaba** d'escriure's la còpia de seguretat, no quan va començar.

Exemple: Si la còpia comença a les 10:00 i acaba a les 10:10, el fitxer .tar tindrà data 10:10. Si un usuari modifica un fitxer a les 10:05 (mentre es feia la còpia), la següent còpia incremental buscarà

fitxers posteriors a les 10:10. Per tant, **el canvi realitzat a les 10:05 es perdrà i no s'inclourà en la incremental.**

Solució: Es pot resoldre creant un "fitxer de marca de temps" (fitxer testimoni) just **abans** de començar la còpia:

Creem marca: `touch /tmp/timestamp_inici`

Fem el backup.

A la següent incremental, usem `--newer=/tmp/timestamp_inici` en lloc del fitxer `.tar`.

Realitzeu una segona ronda de modificacions al directori `/root` per tal de provocar una segona còpia incremental:

Modificacions:

- Genereu nous fitxers
- Modifiqueu el contingut d'alguns fitxers
- Useu la comanda `touch` per canviar la data de modificació d'alguns fitxers.
- Esborreu algun dels fitxers que heu generat per la primera còpia incremental anterior.

Realitzeu una segona còpia incremental del directori `/root` (respecte la primera còpia incremental) amb la comanda **tar**. També feu un **sha512sum** de la segona còpia incremental i poseu-li un nom apropiat.

Segona ronda de modificacions

```
root@davidP (dl. de des. 08):/home/homeA# touch /root/nou_fitxer_2.txt
root@davidP (dl. de des. 08):/home/homeA# echo "Més canvis segona ronda" | tee -a /root/nou_fitxer_1.txt
Més canvis segona ronda
root@davidP (dl. de des. 08):/home/homeA# touch /root/nou_fitxer_1.txt
root@davidP (dl. de des. 08):/home/homeA# rm /root/nou_fitxer_1.txt
```

Segona còpia incremental i Checksum

```
root@davidP (dl. de des. 08):/home/homeA# tar -cvf backup_inc2.tar --newer=backup_inc1.tar /root
root@davidP (dl. de des. 08):/home/homeA# sha512sum backup_inc2.tar > backup_inc2.asc
root@davidP (dl. de des. 08):/home/homeA#
```

Com es pot verificar que el contingut del fitxer de *backup* sigui el mateix que el directori que s'ha copiat?

```
root@davidP (dl. de des. 08):/home/homeA# tar --diff -f backup_inc2.tar -C /
root@davidP (dl. de des. 08):/home/homeA#
```

Si no surt res per pantalla, és que són idèntics. Si hi ha diferències (per exemple, fitxers que han canviat des del backup), les llistarà. En aquest cas son idèntics.

7

Com es pot verificar, fent ús de la comanda **sha512sum**, la integritat d'una còpia de seguretat, és a dir, que el fitxer no ha estat modificat des que es va realitzar la còpia?

```
root@davidP (dl. de des. 08):/home/homeA# sha512sum -c backup_inc2.asc
backup_inc2.tar: CORRECTE
```

3.3. Restauració d'una còpia de seguretat

Reanomenem el directori `/root` per `/root.old` per simular l'efecte que es produiria si esborréssim el directori.

`sudo mv /root /root.old`

```
root@davidP (dl. de des. 08):/home/homeA# mv /root /root.old
root@davidP (dl. de des. 08):/home/homeA#
```

En quin ordre cal restaurar els fitxers per tal que el resultat final sigui el desitjat?

L'ordre ha de ser CRONOLÒGIC (del més antic al més nou).

Primer: **Còpia Completa** (Base).

Segon: **Còpia Incremental 1.**

Tercer: **Còpia Incremental 2.**

Per què? `tar` simplement sobreescrui fitxers quan extreu.

Si restaures primer la incremental (més nova) i després la completa (més vella), la completa sobreescrirà les versions noves dels fitxers amb les versions velles. En fer-ho cronològicament, assegures que l'última versió aplicada sobre el disc és la més recent.

Ara restaureu la còpia de seguretat del directori /root, la qual cosa implica restaurar els tres fitxers que hem creat: la còpia completa i les dos incrementals.

1. Restaurem la base `sudo tar -xvf backup_completa.tar -C /`

2. Apliquem la primera capa de canvis `sudo tar -xvf backup_inc1.tar -C /`

3. Apliquem la segona capa de canvis `sudo tar -xvf backup_inc2.tar -C /`

Què ha passat amb els fitxers que havíeu esborrat abans de fer la segona còpia incremental? Com es pot detectar que aquest fitxers han estat esborrats? Quan seran esborrats de les còpies de seguretat?

Aquí és on es veu la limitació de fer incrementals amb l'opció `--newer`.

Què ha passat amb el fitxer que havies esborrat? El fitxer **ha tornat a aparèixer (s'ha restaurat)**, encara que l'havies esborrat abans de la segona còpia.

Per què?

El fitxer existia a la `backup_inc1.tar`. Quan l'has restaurat (pas 2), s'ha creat al disc.

Quan vas fer la `backup_inc2.tar`, com que el fitxer ja no existia, simplement no es va incloure al tar.

Però l'opció `--newer` de tar **no guarda informació sobre fitxers esborrats**, només sobre fitxers modificats o creats. Per tant, en restaurar la `inc2`, aquesta no li diu al sistema "esborra aquest fitxer", simplement no fa res amb ell. Resultat: el fitxer vell de la `inc1` es queda allà.

Com es pot detectar que aquests fitxers han estat esborrats? Amb aquest mètode de backup (`--newer`), **no es pot detectar automàticament durant la restauració**. L'única manera seria comparar manualment el directori restaurat amb l'estat actual (si encara existís, per exemple `/root.old`) usant: `diff -r /root /root.old`

Quan seran esborrats de les còpies de seguretat? Mai s'esborraran automàticament dels fitxers `.tar` antics. Aquests fitxers "zombis" desapareixeran de la restauració només quan **faci una nova Còpia Completa (Full Backup)**. Aquesta nova còpia completa llegirà l'estat actual del disc (on el fitxer ja no hi és) i, per tant, serà una còpia neta.

Nota tècnica: Per solucionar això en entorns reals amb tar, s'utilitza l'opció **`--listed-incremental` (o `-g`)**, que crea un fitxer de metadades (snapshot) que sí registra els esborrats.

8

3.4. Restauració d'un fragment

Reanomenem un dels subdirectoris dins del directori /root per simular l'efecte que es produiria si esborréssim el subdirectori.

```
root@davidP (dl. de des. 08):/home/homeA# mv /root/nou_directori_1 /root/nou_directori_1.bak
root@davidP (dl. de des. 08):/home/homeA#
```

Restaureu només aquest directori a partir de la còpia de seguretat que hem fet amb **tar**. Això implica restaurar únicament aquest subdirectori a partir dels tres fitxers que hem creat: la còpia completa i les dos incrementals.

```
root@davidP (dl. de des. 08):/home/homeA# tar -xvf backup/backup-root-nivell0-20251208204355.tar.gz --wildcards '/root/nou_dir
ectori_1.bak/' -C /tmp/root
```

4. Realització de còpies usant RSYNC

Fins ara hem vist com fer *backups* i guardar-los en la mateixa màquina, però el més comú és tenir diferents màquines en què fem *backups* i una altra màquina en xarxa que emmagatzema aquests *backups*. Per fer això tenim la comanda **rsync** que permet copiar un directori (o un conjunt de fitxers) a un altre directori a través de una connexió de xarxa. Per això **rsync** usa un algorisme de *checksum* eficient per transmetre només les diferències entre els dos directoris i al mateix temps comprimeix els fitxers per fer més ràpida la transmissió de dades. Aquesta eina permet copiar fitxers des de o cap a un directori situat en una màquina remota, o entre directoris de la mateixa màquina. El que no permet és copiar directoris entre dos màquines remotes. A més a més **rsync** permet copiar enllaços, dispositius, i preservar permisos, grups i propietaris. També suporta llistats d'exclusió i connexió remota amb *shell* segur (ssh) entre altres possibilitats. Per a més

informació veure **man rsync**.

```
root@davidP (dj. d'oct. 23):~# apt-get install rsync
S'està llegint la llista de paquets... Fet
S'està construint l'arbre de dependències... Fet
S'està llegint la informació de l'estat... Fet
El paquets següents s'han instal·lat automàticament i ja no són necessaris:
bsdmainutils g++-10 libaom0 libapt-pkg6.0 libcbor0 libdav1d4 libdns-export1110 libflac8 libicu63 libid
libisc-export1105 libmpdec2 libmpdec3 libnsl-dev libopenexr23 libperl5.28 libperl5.32 libprocps8 libpy
libpython3.7-stdlib libpython3.9-minimal libpython3.9-stdlib libreadline7 libruby2.5 libstdc++-10-dev
libx265-165 libx265-192 linux-image-4.19.0-6-amd64 perl-modules-5.28 perl-modules-5.32 python3.7-minim
python3.9-minimal ruby2.7
Empreu «apt autoremove» per a suprimir-los.
Paquets suggerits:
openssh-server python3-braceexpand
S'instal·laran els paquets NOUS següents:
rsync
0 actualitzats, 1 nous a instal·lar, 0 a suprimir i 48 no actualitzats.
S'ha d'obtenir 428 kB d'arxius.
Després d'aquesta operació s'utilitzaran 828 kB d'espai en disc addicional.
Bai:1 http://ftp.es.debian.org/debian stable/main amd64 rsync amd64 3.4.1+ds1-5 [428 kB]
S'ha baixat 428 kB en 1s (567 kB/s)
S'està seleccionant el paquet rsync prèviament no seleccionat.
(S'està llegint la base de dades... hi ha 124268 fitxers i directoris instal·lats actualment.)
S'està preparant per a desempaquetar ../rsync_3.4.1+ds1-5_amd64.deb...
S'està desempaquetant rsync (3.4.1+ds1-5)...
S'està configurant rsync (3.4.1+ds1-5)...
rsync.service is a disabled or a static unit, not starting it.
S'estan processant els activadors per a man-db (2.9.4-2)...
root@davidP (dj. d'oct. 23):~#
```

9

4.1. Realització de backups a través d'una xarxa

Com ja hem dit abans, **rsync** permet fer còpies en una màquina remota. Això es podria fer utilitzant **rsh** o posant **rsync** en mode servidor, però això pot ser perillós perquè en una xarxa local alguna altra màquina podria estar "escoltant" la connexió i podria agafar informació confidencial. Per aquesta raó, **rsync** permet fer connexions segures amb **ssh**. Per realitzar les nostres proves utilitzarem la nostra pròpia màquina com servidor ssh. En primer lloc inicialitzeu el *daemon* de **ssh**.

Creeu un directori per fer les còpies en la partició de *backups* i després feu la següent comanda:

```
$ rsync -avz /root -e ssh root@localhost:/backup/backup-rsync/
```

Nota: Perquè l'anterior comanda funcioni bé és necessari activar el compte del root i posar-li una contrasenya vàlida. Recordeu instal·lar també el paquet ssh si us cal.

Quin és el significat de les opcions -avz de l'**rsync**?

Aquestes són les banderes (flags) més habituals de l'**rsync**. Aquí tens què fa cadascuna:

-a (Archive mode): És la més important. És una opció "contenedor" que equival a activar moltes opcions alhora (-rlptgoD).

Fa la còpia **recursiva** (copia subdirectoris).

Preserva gairebé tot: enllaços simbòlics, permisos, data de modificació, grup i propietari (owner). Garanteix que la còpia sigui fidel a l'original.

-v (Verbose): Mode "xerraire". Mostra per pantalla quins fitxers s'estan copiant en aquell moment. Sense això, l'**rsync** treballaria en silenci.

-z (Compress): Comprimeix les dades **abans** d'enviar-les per la xarxa i les descomprimeix al destí. Això estalvia ample de banda (fa que la transferència sigui més ràpida si la xarxa és lenta), tot i que gasta una mica més de CPU.

Creeu un arxiu en el directori root amb:

```
$ echo "nou arxiu" > /root/arxiu_nou.txt
```

i torneu a fer el mateix **rsync** d'abans.

```
root@davidP (dl. de des. 08):~# echo "nou archiu" > /root/arxiu nou.txt
```

Ara esborreu l'arxiu que heu creat abans i torneu a sincronitzar.

```
root@davidP (dl. de des. 08):~# rm /root/arxiu_nou.txt
```

Què ha passat amb el fitxer esborrat?

El fitxer **encara existeix al directori de destí** (/backup/backup-rsync/root/arxiu_nou.txt).

Per què? Per defecte, rsync té un comportament **additiu**. Això vol dir que copia fitxers nous i actualitza els modificats, però **no esborra res** al destí encara que hagi desaparegut de l'origen. És una mesura de seguretat per evitar pèrdues de dades accidentals si l'usuari s'equivoca.

10

Amb quin paràmetre podríeu sincronitzar exactament els dos directoris?

Amb aquest:

```
rsync -avz --delete /root -e ssh root@localhost:/backup/backup-rsync/
```

Com faríeu per copiar tots el arxius del directori /home excepte els que tenen extensió .txt?

Per excloure fitxers segons un patró (com l'extensió), s'utilitza l'opció **--exclude**.

La comanda seria:

```
rsync -avz --exclude '*.txt' /home /directori_desti
```

Quina diferència hi ha entre fer `rsync /source /destí` i `rsync /source/ /destí/?`

Aquesta és la font d'errors més comuna en rsync. La diferència rau en la **barra final** (trailing slash):

A) Sense barra final (/source)

Significat: "Agafa la CARPETA source i copia-la dins del destí".

Resultat: Es crea un nou subdirectori al destí.

Si el destí és /backups, tindràs: /backups/source/fitxer1.

B) Amb barra final (/source/)

Significat: "Agafa el CONTINGUT de source i bolca'l dins del destí".

Resultat: Els fitxers es barregen directament al directori de destí.

Si el destí és /backups, tindràs: /backups/fitxer1.

Resum visual:

```
rsync /carpeta /desti -> /desti/carpeta/fitxers... (Copia el contenidor)
```

```
rsync /carpeta/ /desti -> /desti/fitxers... (Copia el contingut)
```

4.2. Realització de còpies incrementals inverses

Com hem vist a l'apartat anterior, cada vegada que realitzem una còpia i sincronitzem, el directori en què tenim el *mirròr* queda exactament igual que el directori d'origen. Això és un problema perquè no tenim control dels canvis realitzats. Per solucionar aquest problema podem utilitzar l'opció **--backup** i **--backup-dir**. Els *backups* generats amb les opcions **--backup** i **--backup-dir** es diuen inversos perquè la còpia completa es la més recent i no la més antiga com amb **tar**. Amb aquesta opció la còpia completa correspon a la última data en que s'ha fet el *backup* i les incrementals a les dels dies anteriors.

A continuació teniu un script senzill per fer *backups* incrementals amb **rsync**.

Completeu-lo amb les dades que facin falta.

11

```
#!/bin/bash
```

```
# Directori que volem copiar (Origen)
```

```
SOURCE_DIR=/root/
```

```
# Directori base on es guardaran les còpies al destí
```

```
DEST_DIR=/backup/backup-rsync
```

```
# Fitxer amb la llista d'exclusions (crea'l encara que estigui buit)
```

```
EXCLUDES=/root/llista_exclosos.txt
```

CRUZ

Nom o IP de la màquina servidor (en aquest cas, la mateixa màquina)
BSERVER=localhost

Format de data: Any-Mes-Dia_Hora-Minut-Segon
BACKUP_DATE=\$(date +%Y-%m-%d_%H-%M-%S)

Opcions per rsync
--backup: activa el mode de còpia de seguretat
--backup-dir: diu on moure els fitxers que han canviat o s'han esborrat
OPTS="--ignore-errors --delete-excluded --exclude-from=\$EXCLUDES \
--delete --backup --backup-dir=\$DEST_DIR/\$BACKUP_DATE -avz"

La transferència real
Això crearà un directori 'complet' que SEMPRE tindrà la versió més recent
rsync \$OPTS \$SOURCE_DIR root@\$BSERVER:\$DEST_DIR/complet/
Ara creeu un fitxer **arxiu.txt** i feu-lo servir per comprovar el funcionament de l'script anterior 12

echo "Versió 1" > /root/arxiu.txt

```
GNU nano 8.4                copia_incremental.sh *
#!/bin/bash

SOURCE_DIR=/root/
DEST_DIR=/backup/backup-rsync
EXCLUDES=/root/llista_exclusos.txt
BSERVER=localhost
BACKUP_DATE=$(date +%Y-%m-%d_%H-%M-%S)
OPTS="--ignore-errors --delete-excluded --exclude-from=$EXCLUDES \ --delete --backup --backup-dir=$DEST_DIR/$BACKUP_DATE -avz"
rsync $OPTS $SOURCE_DIR root@$BSERVER:$DEST_DIR/complet/

root@davidP (dl. de des. 08):~# ./copia_incremental.sh

</backup/backup-rsync/complet># ls
ls
    backup      sudousers
arxiu.txt      backups      usuarios-20231212171129.txt
12binary_install private.key  usuarios-20231212172100.txt
install.sh     sudoers
```

Després modifiqueu aquest **arxiu.txt** i torneu a sincronitzar.

echo "Modificació al arxiu a Documents" > /root/arxiu.txt

</backup/backup-rsync/complet/Documents># cat arxiu.txt

MODIFICACIÓ al arxiu a Documents

Finalment esborreu el fitxer **arxiu.txt** i feu una sincronització més.

rm /root/arxiu.txt

Què observeu en modificar un fitxer i fer una sincronització? I en esborrar-lo?

Quan modifiqueu arxiu.txt a l'origen i executeu l'script, observeu el següent:

Al directori complet (Destí principal): El fitxer s'actualitza a la **nova versió**. Aquest directori sempre actua com un mirall exacte de l'estat actual del teu sistema.

Al directori \$BACKUP_DATE (Directori incremental): Apareix una còpia de l'**antiga versió** del fitxer (la que hi havia abans de modificar-lo).

Per què passa això? Rsync detecta que el fitxer ha canviat. Abans de sobre escriure la versió vella al destí complet, la "salva" movent-la al directori que has especificat amb la data. Així no perds la dada antiga.

Quan esborres arxiu.txt a l'origen i executeu l'script:

Al directori complet (Destí principal): El fitxer **desapareix**. Com que l'hem esborrat a l'origen i tenim l'opció `--delete` activada, el mirall reflexa aquesta eliminació.

Al directori \$BACKUP_DATE (Directori incremental): Apareix el fitxer que acabes d'esborrar.

Per què passa això? Rsync detecta que el fitxer ja no existeix a l'origen i l'ha d'eliminar del destí.

Gràcies a l'opció `--backup`, en lloc d'eliminar-lo permanentment, el **mou** al directori de la data.

4.3. Realització de còpies incrementals inverses tipus snapshot

Una possibilitat que dóna **rsync** és fer *backups* incrementals on, utilitzant una propietat dels enllaços durs, és possible fer que les còpies incrementals semblin còpies completes. Per això analitzarem primer algunes propietats dels enllaços durs.

Repàs d'enllaços durs

El nom d'un fitxer no representa el fitxer mateix, per al sistema és només un enllaç dur al *inode*. Això permet que un fitxer (inode) pugui tenir més d'un enllaç dur. Per exemple si tens un fitxer `file_a` es pot crear un enllaç cridat `file_b`:

`$ ln file_a file_b`

Amb la comanda **stat** es pot saber quants enllaços durs té un fitxer:

`$ stat filename`

Com es pot comprovar que `file_a` i `file_b` pertanyen al mateix *inode*?

Utilitzant la comanda **ls** amb l'opció `-i` (inode) o la comanda **stat**.

`ls -li file_a file_b`

13

Què passa amb el fitxer `file_b` si es fa un canvi a `file_a`?

Si edites el contingut de `file_a` (per exemple, afegint text), **file_b canvia automàticament**.

Per què? Perquè no són dos fitxers diferents sincronitzats. Són **dues "etiquetes" (noms)**

enganxades al mateix contenidor de dades (inode). Si canvies el contingut del contenidor, es veu el canvi des de qualsevol de les etiquetes.

I si es fa un canvi de permisos a `file_a`?

Si fas un `chmod 777 file_a`, els permisos de `file_b` **també canviaran**.

Per què? Els permisos, el propietari i la data de modificació es guarden a les metadades de l'**inode**, no al nom del fitxer. Com que tots dos apunten al mateix inode, comparteixen els mateixos permisos.

I si copiem un altre fitxer sobreescrivint el fitxer `file_a` (**cp file_c file_a**)?

Si fas una còpia estàndard sobreescrivint un enllaç dur:

Resultat: El contingut de `file_c` es bolca dins de l'espai de dades apuntat per `file_a`.

Efecte en file_b: Com que `file_b` apunta al mateix espai, **file_b també canvia** i ara tindrà el contingut de `file_c`.

Inode: L'inode es manté, només canvien les dades del disc.

I si es sobre escriu amb l'opció **--remove-destination**?

Aquí és on la cosa canvia radicalment. Si fas: `cp --remove-destination file_c file_a`

El sistema primer **esborra** (unlink) l'enllaç `file_a`.

Després crea un **nou fitxer** `file_a` (amb un **nou inode**) i hi posa el contingut de `file_c`.

Efecte en file_b:

`file_b` **NO canvia**. Segueix apuntant a l'inode vell (amb les dades velles).

Ara `file_a` i `file_b` són fitxers totalment independents.

Això és clau per trencar l'enllaç quan volem modificar un fitxer en un backup sense alterar les còpies antigues.

I què passa amb `file_b` si `file_a` és esborrat?

Si fas `rm file_a`:

Resultat: `file_b` **segueix existint i accessible**. Les dades no es perden.

Per què?

Esborrar (rm) en Linux realment significa "desvincular" (unlink). El sistema només allibera l'espai del disc (l'inode) quan el **comptador d'enllaços arriba a 0**. Si tenies 2 enllaços i n'esborres 1, el comptador passa a 1. L'inode i les dades es mantenen vius gràcies a file_b.

La comanda **cp** té una opció per fer còpies en què realment no es fa una còpia sinó un enllaç dur (**cp -l**). Una altre opció interessant es **-a** que fa una còpia recursiva i preserva els permisos de accés, els temps i els propietaris dels fitxers

4.3.1 Backups tipus "snapshot" amb rsync i cp -al

Es poden combinar **rsync** i **cp -al** per crear *backups* que semblin múltiples còpies completes d'un sistema de fitxers sense que sigui necessari gastar tot l'espai en disc requerit per totes les còpies, en resum es podria fer:

14

```
rm -rf backup.3
mv backup.2 backup.3
mv backup.1 backup.2
cp -al backup.0 backup.1
rsync -a --delete source_directory/ backup.0/
```

Si les comandes anteriors s'executen cada dia, backup0, backup1, backup2 i backup3 apareixeran com si fossin còpies completes del directori source_directory com estava avui, ahir, abans d'ahir i tres dies abans respectivament. Però en realitat l'espai extra serà igual a la grandària del directori source_directory més la grandària total dels canvis dels últims tres dies. Exactament el mateix que un *backup* complet més els *backups* incrementals con heu fet abans amb **tar** i el mateix **rsync**. L'únic problema és que els permisos i propietaris de les còpies anteriors serien els mateixos que els de la còpia actual.

Hi ha una opció d'**rsync** que fa directament la còpia amb enllaços durs (**--link-dest**) i d'aquesta manera no seria necessari utilitzar la comanda **cp**, a més a més que preserva els permisos i propietaris de les còpies anteriors. Amb aquesta opció l'esquema plantejat anteriorment quedaria així:

```
rm -rf backup.3
mv backup.2 backup.3
mv backup.1 backup.2
mv backup.0 backup.1
rsync -a --delete --link-dest=../backup.1 \
    source_directory/ backup.0/
```

4.4. Script per fer backups tipus snapshot

Per fer *backups* del directori /root utilitzarem l'script **backup-rsync-snapshot.sh**, que està disponible a l'apèndix.

En primer lloc modifiqueu les variables de la secció "file locations" per posar els valors adequats del vostre sistema.

Sección "File Locations" (Variables)

None

```
MOUNT_DEVICE="/dev/sda4"
SNAPSHOT_MOUNTPOINT="/backup"
SNAPSHOT_DIR="root-backup"
EXCLUDES="/root/exclude_list.txt"
SOURCE_DIR="/root"
```


Després completeu la secció amb la comanda **rsync** amb els valors apropiats per fer la còpia tipus *snapshot*

```
$RSYNC -a --delete --exclude-from=$EXCLUDES
--link-dest=$SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.1/ $SOURCE_DIR/
$SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.0/
```

Ara feu modificacions als fitxers del directori origen (per exemple crear un nou fitxer, modificar el contingut i la data d'accés a un fitxer o esborrar un fitxer) i torneu a executar l'script. Feu això varies vegades fins que tingueu una còpia actual i tres còpies anteriors.

Què observeu al modificar un fitxer i fer una sincronització?

Quan modifiqueu un fitxer (per exemple fitxer_prova.txt) i executeu l'script, passa el següent:

A daily.1 (Còpia vella): Es conserva el fitxer original tal com estava abans del canvi. Aquest fitxer manté l'**inode** original.

A daily.0 (Còpia nova): rsync detecta que el fitxer ha canviat respecte a l'anterior. En lloc de fer un enllaç dur, **copia el fitxer sencer de nou**.

Resultat: Ara tens dos fitxers físics diferents al disc (amb **inodes diferents**).

Quina és la grandària del directori /backup.0 i dels altres directoris backup1, backup2 i backup3?

Si executeu `du -sh` sobre els directoris de backup.

Observació: Podem veure que **TOTS** els directoris (daily.0, daily.1, daily.2, etc.) semblen tenir la **mateixa grandària** (aproximadament la mida total de /root).

Explicació: Això és una il·lusió causada pels **enllaços durs (hard links)**.

La comanda `du` compta l'espai que ocupen els fitxers dins la carpeta.

Com que el 99% dels fitxers són compartits (són el mateix fitxer físic apuntat des de diferents carpetes), `du` els suma a totes les carpetes.

Realitat: L'espai **real** ocupat al disc és: *(Mida d'una còpia completa) + (Mida dels petits canvis fets cada dia)*.

Finalment, voldríem fer una restauració. Canvieu el nom del directori /root per simular una pèrdua de dades. Feu una restauració d'aquest directori amb la còpia de seguretat més recent. Finalment, per simular el desastre i restaurar la versió més nova (**daily.0**), utilitza aquestes comandes:

Simular pèrdua:

```
mv /root /root.old_desastre
```

Restaurar: Aquí és crucial posar la barra / al final del directori origen per copiar el *contingut* i no crear una carpeta daily.0 dins de /root

```
mkdir /root
```

```
chmod 700 /root
```

```
rsync -axv /backup/root-backup/daily.0/ /root/
```

Verificació: Ara /root té exactament els mateixos fitxers que tenies just abans de l'últim backup.

5. Referències

(1) Lars Wirzenius, Joanna Oja, Stephen Stafford, Alex Weeks, **The Linux System Administrator's Guide** Version 0.9, <http://tldp.org/LDP/sag>

(2) AEleen Frisch, **Essential System Administration**. O'Reilly. 2002.

(3) Mike Rubel. **Easy Automated Snapshot-Style Backups with Linux and Rsync**.

http://www.mikerubel.org/computers/rsync_snapshots/

(4) **Rsnapshot**. *a remote filesystem snapshot utility based on rsync for making backups of local and remote systems*. <http://www.rsnapshot.org/>

6. Apèndix. Codi de l'script per còpies tipus snapshot

None

```
#!/bin/bash
# -----
# mikes handy rotating-filesystem-snapshot utility
# http://www.mikerubel.org/computers/rsync_snapshots
# Modified by Mauricio Alvarez: http://people.ac.upc.edu/alvarez
# -----

# ----- system commands used by this script -----
ID=/usr/bin/id
ECHO=/bin/echo
MOUNT=/bin/mount
RM=/bin/rm
MV=/bin/mv
CP=/bin/cp
TOUCH=/usr/bin/touch
RSYNC=/usr/bin/rsync

# ----- file locations -----
# Ajusta /dev/sda4 al dispositiu on tinguis muntat /backup
MOUNT_DEVICE="/dev/sda4"
SNAPSHOT_MOUNTPOINT="/backup"
SNAPSHOT_DIR="root-backup"
# Assegura't que aquest fitxer existeix, encara que estigui buit
EXCLUDES="/root/exclude_list.txt"
SOURCE_DIR="/root"

# ----- the script itself -----

# make sure we're running as root
if (( `ID -u` != 0 )); then
    $ECHO "Sorry, must be root. Exiting..."
    exit
fi

# attempt to remount the RW mount point as RW; else abort
# Nota: Si /backup ja està muntat com RW, això no farà mal
$MOUNT -o remount,rw $MOUNT_DEVICE $SNAPSHOT_MOUNTPOINT
if (( $? )); then
    $ECHO "snapshot: could not remount $SNAPSHOT_MOUNTPOINT readwrite"
    exit
fi

# rotating snapshots of /$SNAPSHOT_DIR

# step 1: delete the oldest snapshot, if it exists:
if [ -d $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.3 ] ; then
    $RM -rf $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.3
fi

# step 2: shift the middle snapshots back by one, if they exist
if [ -d $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.2 ] ; then
    $MV $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.2
$SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.3
fi

if [ -d $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.1 ] ; then
    $MV $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.1
$SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.2
fi
```

```
# L'antic daily.0 passa a ser daily.1 (aquesta serà la nostra base per link-dest)
if [ -d $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.0 ] ; then
    $MV $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.0
$SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.1
fi

# step 3: rsync from the system into the latest snapshot (daily.0)
# AQUESTA ÉS LA PART QUE FALTAVA:
$RSYNC -av --delete \
    --exclude-from=$EXCLUDES \
    --link-dest=$SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.1 \
    $SOURCE_DIR/ \
    $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.0/

# step 5: update the mtime of daily.0 to reflect the snapshot time
$TOUCH $SNAPSHOT_MOUNTPOINT/$SNAPSHOT_DIR/daily.0

# now remount the RW snapshot mountpoint as readonly
# Nota: Comenta aquestes línies si necessites seguir escrivint a /backup
manualment
$MOUNT -o remount,ro $MOUNT_DEVICE $SNAPSHOT_MOUNTPOINT
if (( $? )); then
    $ECHO "snapshot: could not remount $SNAPSHOT_MOUNTPOINT readonly"
    exit
fi
```